

# Artificial Intelligence: Homework 1 Written

Mu RuoTong,225040014

## Introduction

**INTRO1: Meaning of "Rationally"** In AI, acting "rationally" means choosing actions that are expected to maximize a specific utility function or performance measure, given the agent's current knowledge and available information.

- **Example 1:** A self-driving car choosing to brake rather than swerve into traffic, maximizing the utility of passenger safety.
- **Example 2:** Rational investing — Making investment decisions based on fundamental financial data rather than market hype or emotions.

**INTRO2: Tasks humans do poorly** Humans do not perform well at tasks requiring the rapid, continuous processing of massive datasets. An example is scanning millions of real-time credit card transactions globally to detect subtle patterns of financial fraud.

## Search Algorithms

**BFS1: Breadth-First Search States** Assuming a constant branching factor  $b$ , the number of generated states up to level 5, where half of the states on level 5 are searched, is mathematically represented as:

$$1 + b + b^2 + b^3 + b^4 + \frac{b^5}{2}$$

Based on the exact 8-puzzle diagram provided, the goal node is explicitly labeled as node 27. Therefore, 27 states were searched to reach the goal.

**DFS1: DFS Search Tree** Based on the fractional entry and exit times provided in the graph diagram ( $u : 1/12, v : 2/11, w : 7/8, s : 4/5, t : 3/10, r : 6/9$ ), the DFS traversal perfectly matches the sequence:  $u \rightarrow v \rightarrow t \rightarrow s \rightarrow (\text{backtrack}) \rightarrow r \rightarrow w$ . The tree edges are  $(u, v)$ ,  $(v, t)$ ,  $(t, s)$ ,  $(t, r)$ , and  $(r, w)$ .

**DFS2: Entering and Leave Times** Tracing alphabetical DFS starting from  $S$ , the discovery/finish times are:

- $S : 1/16$
- $A : 2/15$
- $B : 3/14$
- $C : 4/5$

- $E : 6/13$
- $D : 7/8$
- $F : 9/12$
- $G : 10/11$

**DFS3: Directed Graph Alphabetical DFS** Starting at  $q$ , the traversal sequence and backtracking is as follows:

1. Start at  $q$ , visit  $s$ , visit  $v$ , visit  $w$ .
2. Backtrack to  $q$ , visit  $t$ , visit  $x$ , visit  $z$ .
3. Backtrack to  $t$ , visit  $y$ .
4. Backtrack to root. Pick next unvisited:  $r$ .
5. Start at  $r$ , visit  $u$ . All nodes visited.

## Pathfinding

**Dijkstra1** Running Dijkstra's algorithm from node  $a$ , nodes are added to the visited set  $S$  in the following order: 1.  $S = \{a\}$  2.  $S = \{a, b\}, b = 4$  3.  $S = \{a, b, h\}, h = 5$  4.  $S = \{a, b, h, g\}, g = 6$  5.  $S = \{a, b, h, g, f\}, f = 8$  6.  $S = \{a, b, h, g, f, c\}, c = 12$  7.  $S = \{a, b, h, g, f, c, i\}, i = 12$  8.  $S = \{a, b, h, g, f, c, i, e\}, e = 18$  9.  $S = \{a, b, h, g, f, c, i, e, d\}, d = 19$

### A\* 1: Standard Heuristic

- **Iter 1:** Pop  $A(g = 0, f = 30)$ . Open:  $X(f = 44)$ .
- **Iter 2:** Pop  $X(44)$ . Open:  $C(f = 45), B(f = 49)$ .
- **Iter 3:** Pop  $C(45)$ . Open:  $W(f = 48), B(f = 49), L(f = 81)$ .
- **Iter 4:** Pop  $W(48)$ . Open:  $T(f = 48), B(f = 49), K(f = 68), L(f = 81)$ .
- **Iter 5:** Pop  $T(48)$ . Goal reached!

Optimal Path:  $A \rightarrow X \rightarrow C \rightarrow W \rightarrow T$  (Total Cost = 48).

### A\* 2: Modified Heuristic ( $h(B) = 6$ )

- **Iter 1:** Pop  $A(0, 30)$ . Open:  $X(44)$ .
- **Iter 2:** Pop  $X(44)$ . Open:  $B(f = 39), C(f = 45)$ .
- **Iter 3:** Pop  $B(39)$ . Open:  $C(45), T(f = 50), W(f = 54), S(f = 57)$ .
- **Iter 4:** Pop  $C(45)$ . Updates  $W$ . Open:  $W(f = 48), T(50), S(57), L(81)$ .
- **Iter 5:** Pop  $W(48)$ . Updates  $T$ . Open:  $T(f = 48), S(57), K(68), L(81)$ .

- **Iter 6:** Pop  $T(48)$ . Goal!

**Conclusion:** Yes, the same optimal path is found.

**Problem:** Reducing  $h(B)$  to 6 makes the heuristic inconsistent (non-monotonic) because  $h(X) \not\leq c(X, B) + h(B)$  ( $27 \not\leq 16 + 6$ ). Inconsistent heuristics can cause A\* to expand nodes with suboptimal  $g$ -values initially, which can require reopening closed nodes to find the true shortest path, harming performance.

### AStar3: Polygonal Obstacles

- (a) **How many states?** If we use a visibility graph, the continuous coordinate space is reduced to a finite number of states: the start node, the goal node, and the vertices of all polygons.
- (b) **Why along the edge?** In Euclidean geometry, the shortest path between two points is a straight line. A path will only deviate from a straight line when obstructed by an object. Thus, the shortest path must stretch tightly between the obstacle corners (vertices), resulting in straight line segments.